



Machine Learning para detectar zonas urbanas

La teoría del taller, desde cero — percepción remota, geomediana, objetos, aprendizaje automático, y las dos plataformas donde lo ejecutarás.

Abel Alejandro Coronado Iruegas · INEGI

Dirección General de Integración, Análisis e Investigación

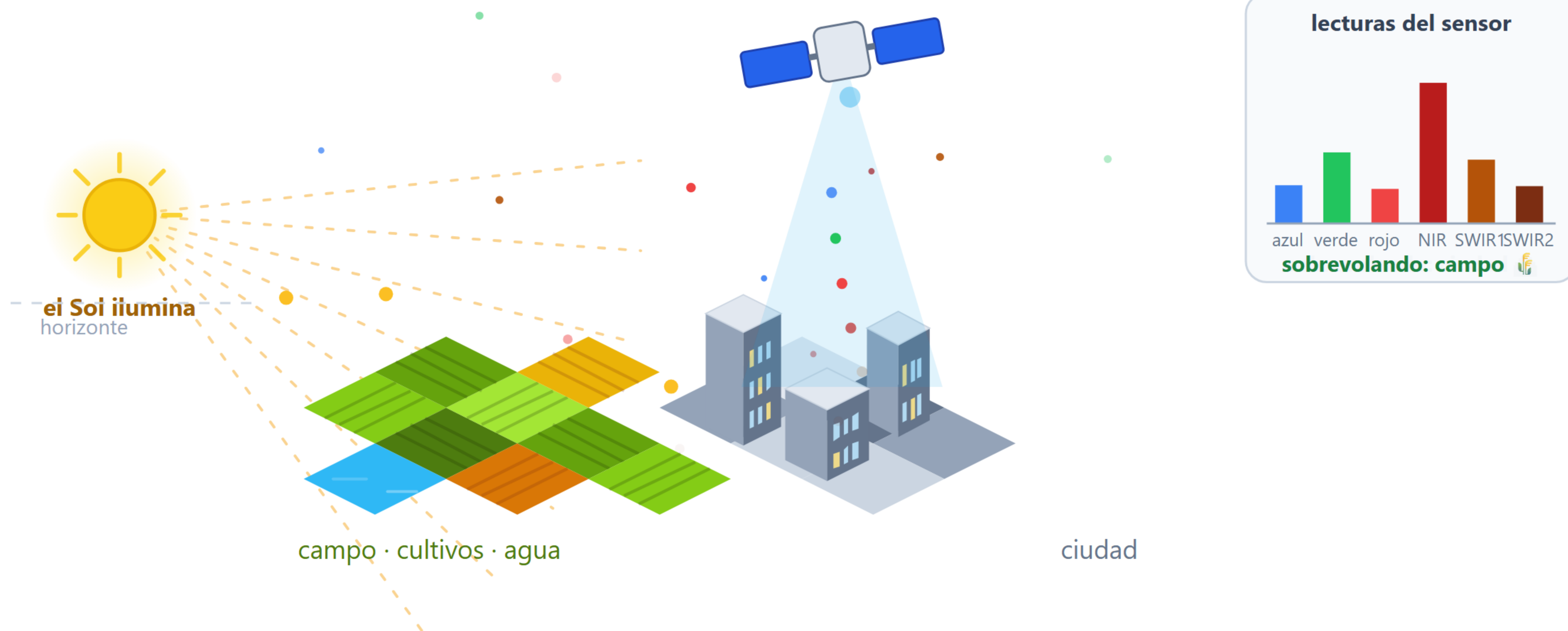
Dirección General Adjunta de Investigación · Laboratorio de Ciencia de Datos

1 ¿Cómo "ve" un satélite?

Todo lo que hay en la superficie — un maizal, un techo de lámina, una presa — **refleja la luz del Sol de manera distinta**. La **percepción remota** consiste en medir esa luz reflejada desde el espacio, sin tocar el territorio.

Usaremos datos de **Sentinel-2** (programa europeo Copernicus): dos satélites que cubren toda la Tierra cada ~5 días con píxeles de 10×10 m, de acceso **público y gratuito**.

sensor PASIVO: el Sol ilumina · el territorio refleja · el satélite mide banda por banda



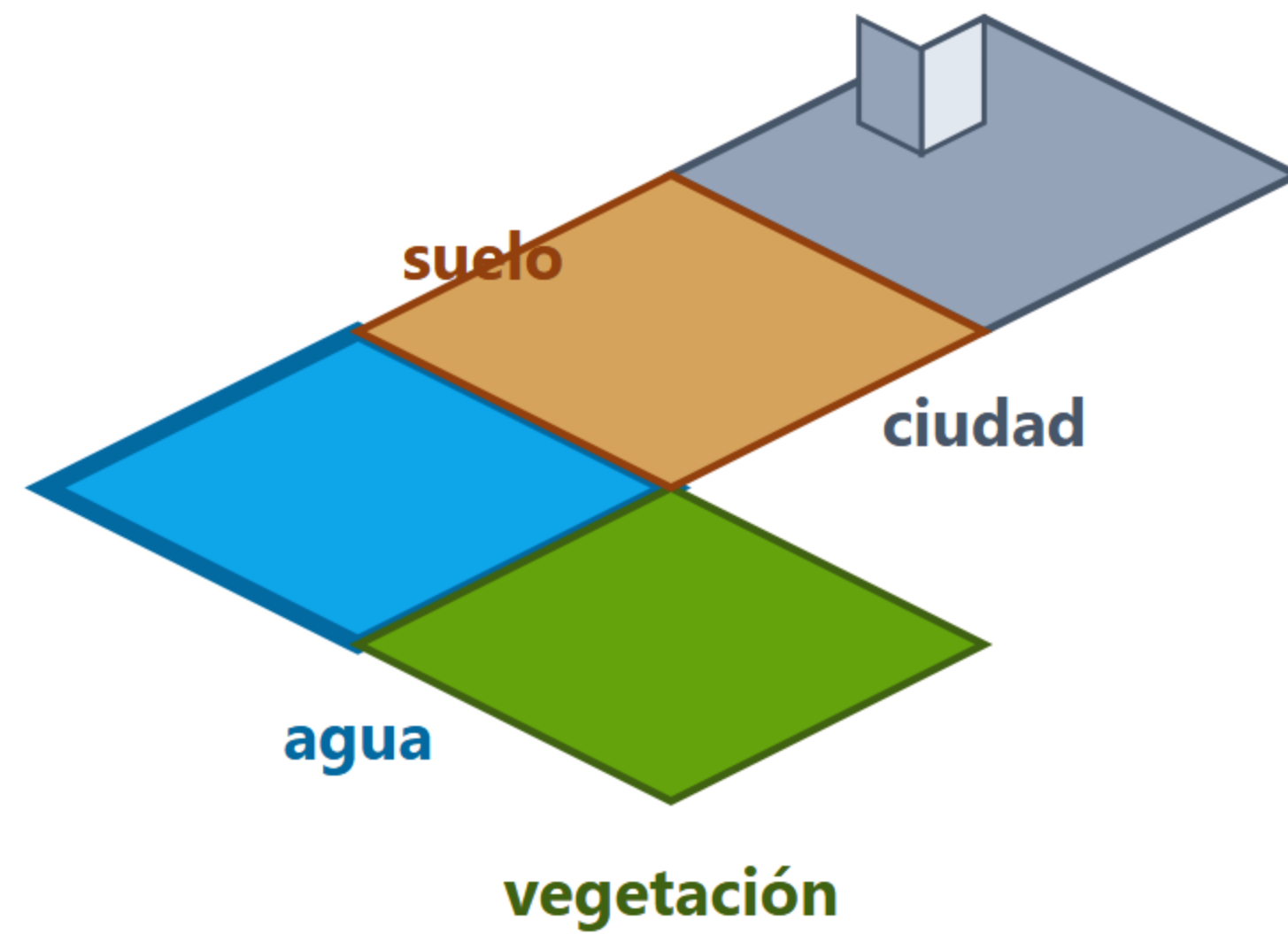
El satélite no emite luz ni toma "fotos": **sensa la intensidad del reflejo solar en varias bandas del espectro** — mira cómo sus lecturas cambian entre el campo (NIR alto) y la ciudad (rojo y SWIR altos).

2 El píxel no es un color: es una firma espectral

Nuestros ojos ven 3 bandas (rojo, verde, azul). Sentinel-2 registra **12 bandas espectrales**, incluyendo infrarrojos donde la vegetación brilla, el agua se apaga y el concreto se delata. Cada píxel es una **columna de mediciones**: su **firma espectral** — la "huella digital" de lo que hay en ese pedacito de territorio.

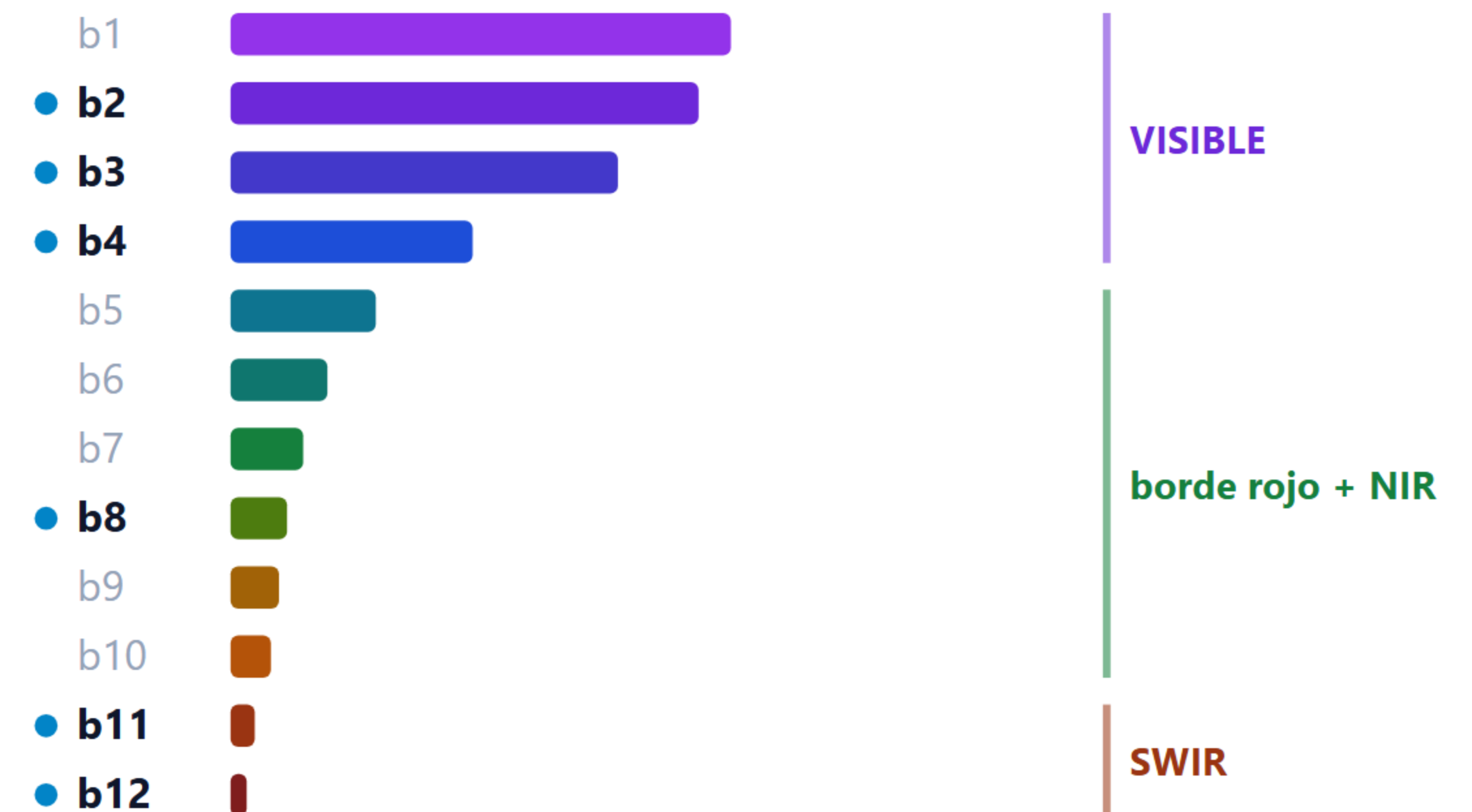
En el taller usarás la versión "**calidad Landsat**" de esa imagen: remuestreada de 10 a **30 metros** y reducida a las **6 bandas clásicas** (azul, verde, rojo, NIR, SWIR1, SWIR2) con las que la familia Landsat lleva 50 años observando la Tierra. La firma se conserva donde importa — los infrarrojos siguen ahí — y la imagen baja de 2.5 GB a 176 MB: lo justo para que **todo el estado de Aguascalientes** quepa en tu navegador.

Esta es la idea más importante del taller: **el modelo de machine learning no verá colores, verá firmas.**



haz clic en un parche 🖱️

firma espectral del agua



• = las 6 bandas "calidad Landsat" que usarás en el taller

el agua absorbe casi todo el infrarrojo: su firma se desploma tras b4 📉

↑ interactivo: compara las cuatro firmas — y nota cómo suelo y ciudad casi se confunden: ese es el problema que tu modelo resolverá

2b La imagen es un cubo: espacio × bandas × tiempo

Solemos imaginar una imagen satelital como una **lámina plana**, pero no lo es. Cada píxel guarda **un número por banda** — así que la imagen tiene **profundidad espectral**: es una pila de capas (las 6 bandas), no una sola. Y como el satélite vuelve a pasar cada cierto tiempo, hay **una toma por fecha**: la **resolución temporal** apila esas tomas en una serie.

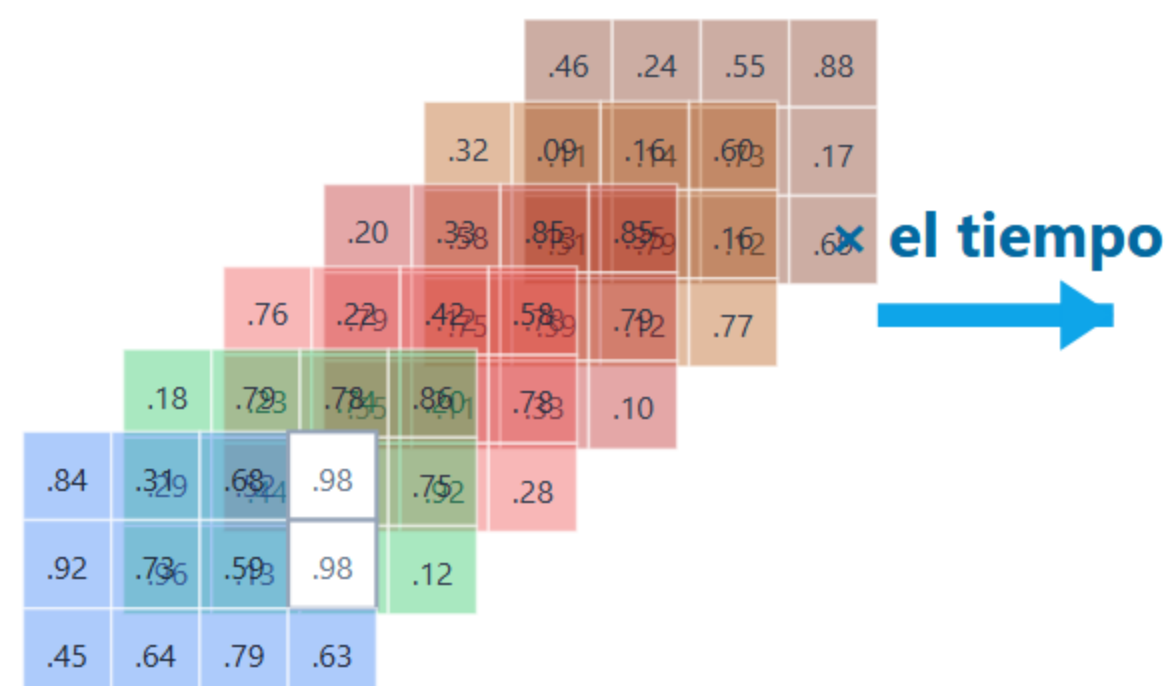
El resultado es un **cubo de datos** (ancho × alto × bandas × tiempo). Algunas fechas traen **nubes** sobre algunos píxeles — por eso después la geomediana (figura 3) resume esa serie quedándose con lo representativo y descartando las nubes.

Una imagen satelital es un CUBO: ancho × alto × bandas — y se repite en el tiempo

la vemos como una lámina, pero cada píxel es una columna de números (una por banda) y cada fecha es otra toma

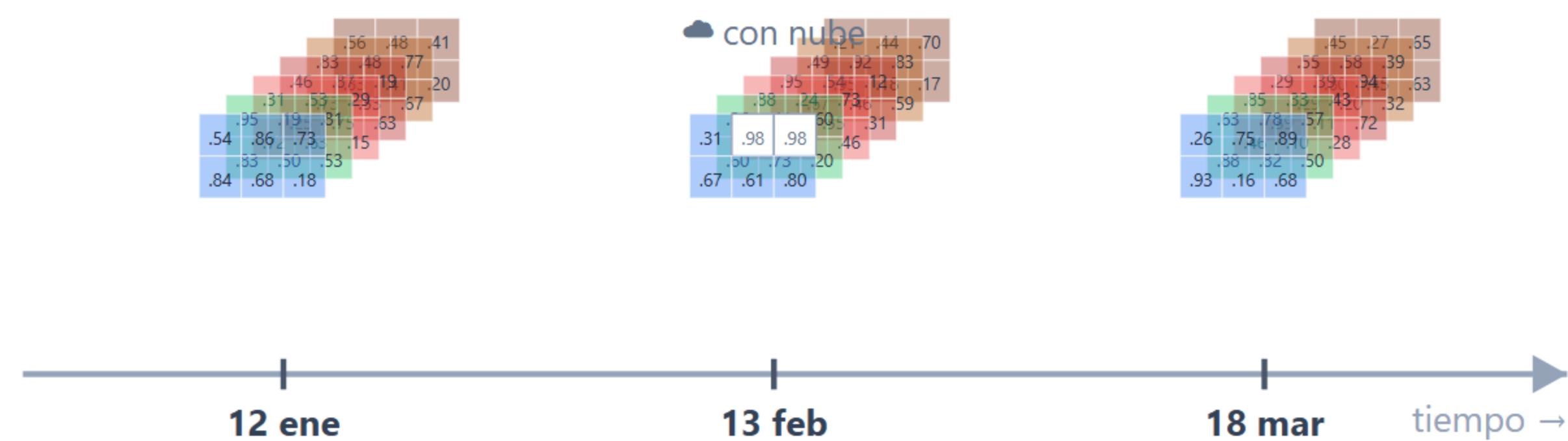
6 bandas:

- b1 azul
- b2 verde
- b3 rojo
- b4 NIR
- b5 SWIR1
- b6 SWIR2



1 toma = un cubo de 6 bandas

color de banda + valor (0–1) · = nube



la resolución temporal apila tomas en el tiempo → la serie que la geomédiana resume

Una imagen no es una lámina: es un cubo multiespectral, repetido en el tiempo. La geomédiana colapsará la dimensión temporal en una sola imagen limpia.

3 La geomédiana: un año de imágenes en una sola, sin nubes

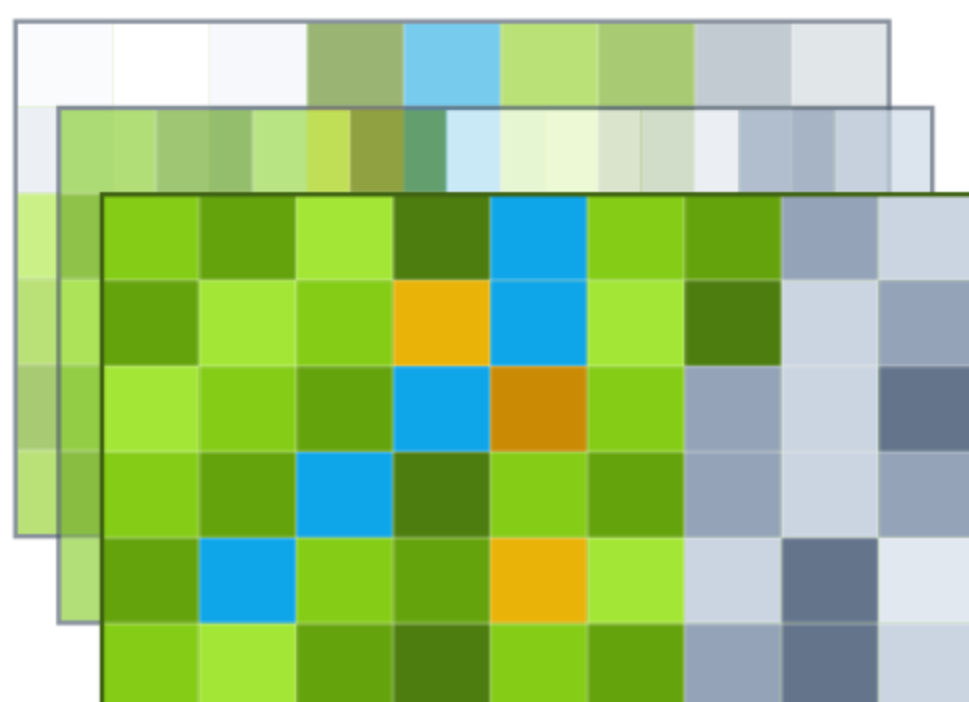
El día de la pasada puede haber **nubes, sombras o bruma**. La solución es un pipeline de cuatro pasos (el mismo que INEGI documenta para su **Geomédiana Nacional** del Cubo de Datos Geoespaciales de México):

1. **La pila:** se reúnen *todas* las pasadas del periodo — para cada píxel, eso es una **serie temporal de vectores** de 6 bandas.
2. **Máscara de calidad:** con las bandas de calidad de la agencia espacial se **descartan las observaciones de nube, sombra y saturación**. La geomédiana además es *robusta*: tolera casi 50 % de observaciones ruidosas — la máscara solo le facilita el trabajo.
3. **La geomédiana de cada píxel** (Roberts et al., 2017): el punto del espacio espectral que **minimiza la distancia a todas las observaciones**. No se puede despejar con una fórmula: se *camina* hacia ella con el **algoritmo de Weiszfeld** — arranca en el promedio y da pasos de promedio pesado (peso = 1/distancia) hasta converger. Crucial: opera sobre **las 6 bandas A LA VEZ**, así que **preserva las relaciones entre bandas** — por eso el resultado sirve para NDVI, NDBI y aprendizaje automático, que viven de esas relaciones.
4. **El mosaico:** los píxeles compuestos se integran en tiles de 5000×5000 y se ensamblan con GDAL — consistencia espacial incluso entre escenas.

*La geomédiana de cada píxel es el punto del espacio espectral que **minimiza la distancia a todas las observaciones del año**. A diferencia del promedio, los datos atípicos apenas la desplazan.*

La del taller es la `GM_AGS_2020.tif`, geomédiana Sentinel-2 (Coronado et al., 2022 — datos abiertos en Zenodo); la metodología nacional está en el [documento metodológico de INEGI \(2020\)](#).

1· la pila del periodo



cada píxel = serie temporal
de vectores de 6 bandas

2· máscara de calidad



bandas de calidad: fuera nube,
sombra y saturados (robusta hasta ~50 % de ruido)

3· un píxel, de cerca

su serie temporal limpia:

12-ene → [96, 205, 152, ...]

28-ene → [94, 209, 148, ...]

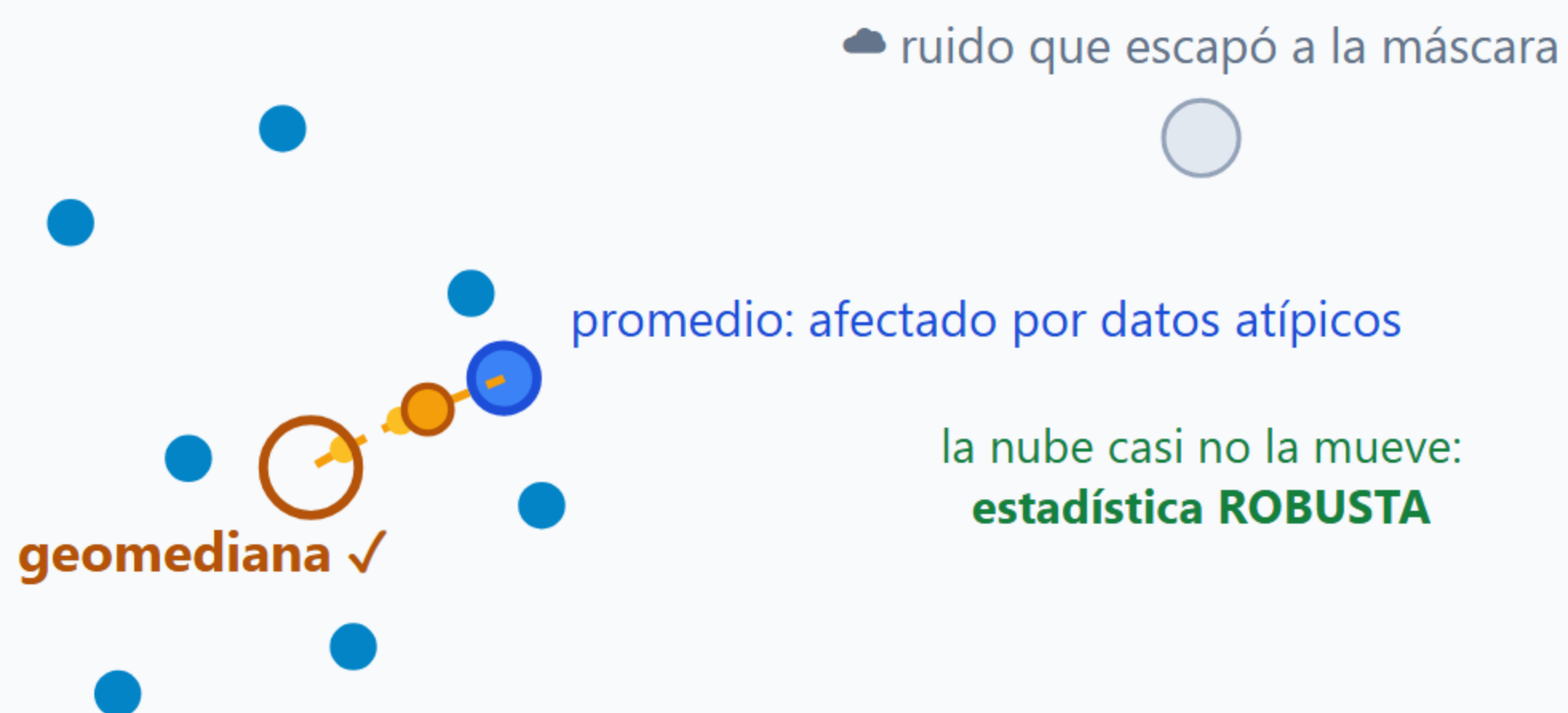
13-feb → [99, 198, 155, ...]

: (todo el periodo)

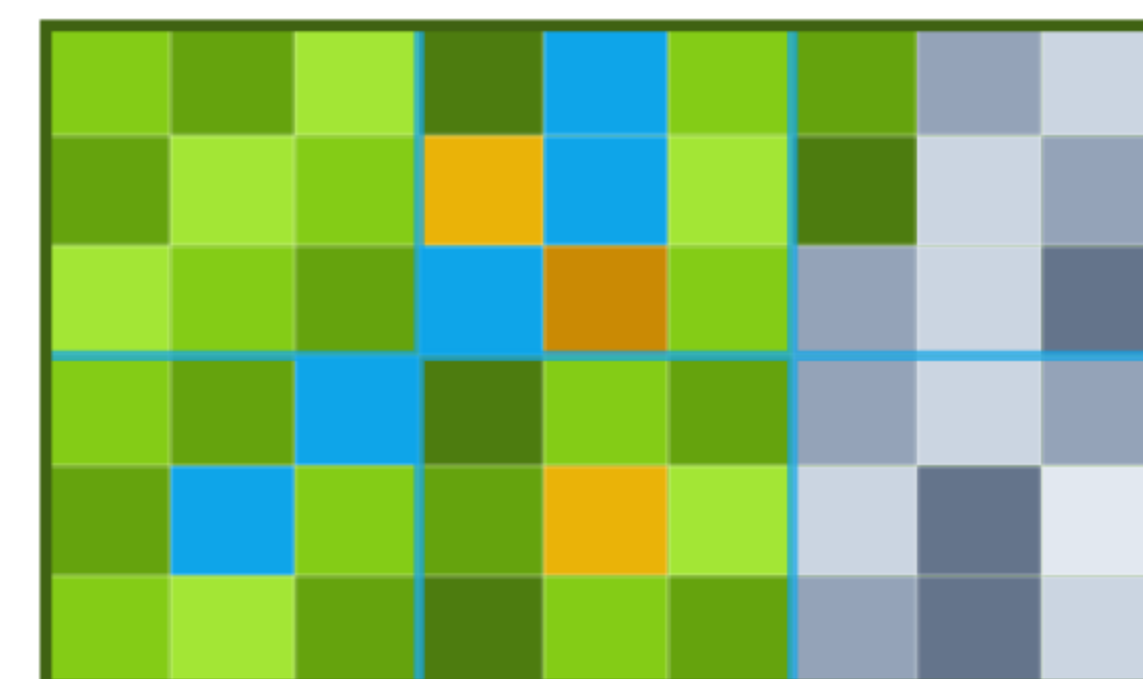
¿cuál lo representa MEJOR?

la respuesta: la GEOMEDIANA — el camino de Weiszfeld

espacio espectral — las 6 bandas A LA VEZ



4· el mosaico limpio



teselas de 3×3 píxeles → mosaico (GDAL)

✓ **sin nubes, consistente entre escenas**

✓ **preserva las relaciones entre bandas**

por eso sirve para NDVI, NDBI y... tu modelo de ML

Geomadiana Nacional INEGI (Cubo de Datos): +118 mil escenas Landsat desde 1984 · 6 bandas a 30 m — las mismas de tu taller · cómputo ≈ 24 h · 35 GB

El pipeline completo de la geomadiana — INEGI (2020), *documento metodológico de la Geomadiana Nacional*; algoritmo de Roberts, Mueller & McIntyre (2017).

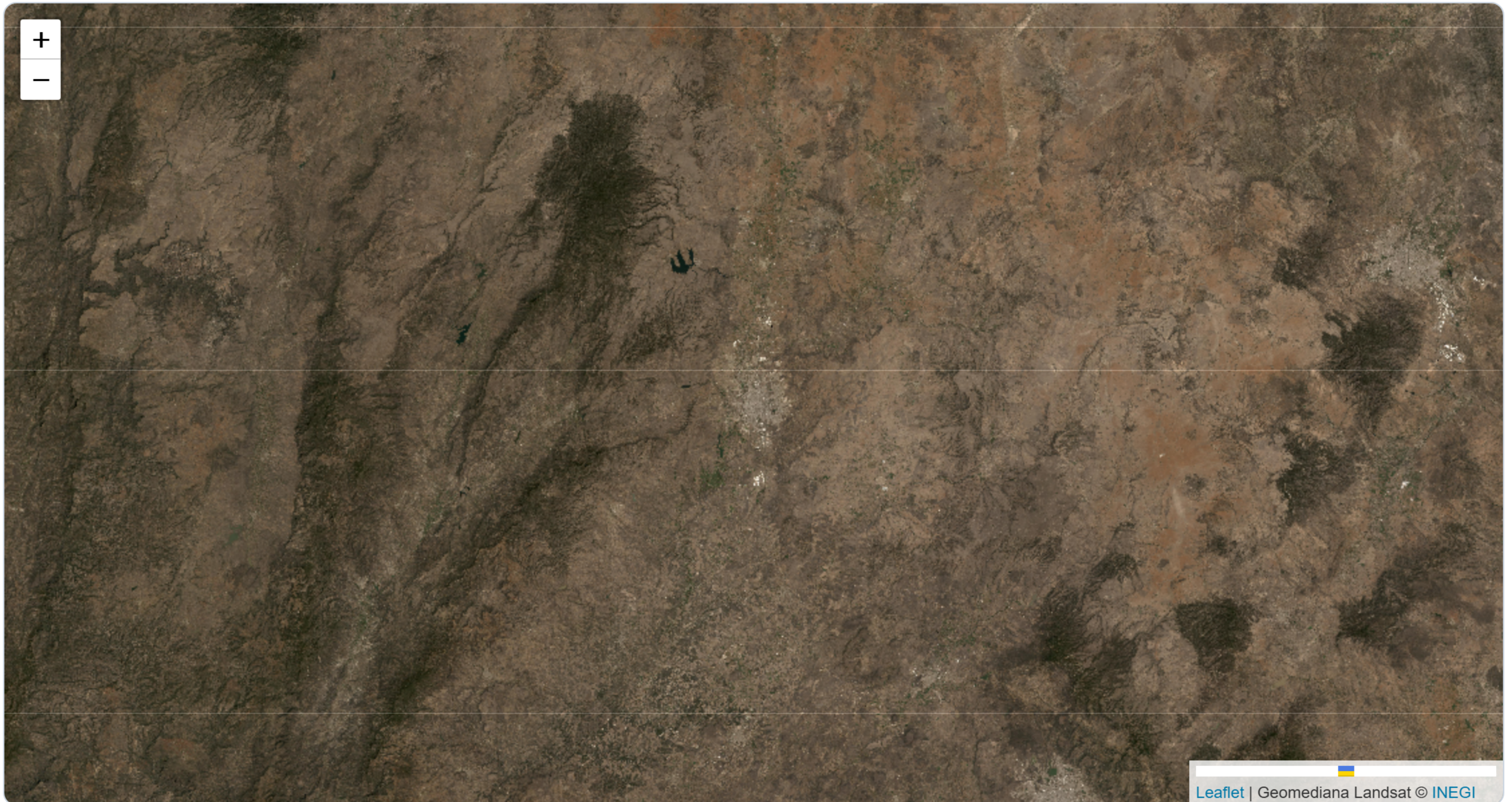
período: 2020 ▼

color natural

falso color

infrarrojo

vs comparar años



📡 Estás navegando las **geomedias Landsat oficiales de México (1984–2024)**, servidas **en vivo** desde los servidores del INEGI (servicio WMS público). Fuente y cita: INEGI, Geomediana — Cubo de Datos Geoespaciales de México.

↑ **viaja en el tiempo: elige un período, cambia la visualización, acércate a tu ciudad — activa  y desliza la cortina para comparar dos años — y maximízalo con  para la clase**

4 Del píxel al objeto: la segmentación

Clasificar píxel por píxel produce mapas "de sal y pimienta". El enfoque profesional (**GEOBIA**) agrupa primero los píxeles en **segmentos** — regiones contiguas y homogéneas que corresponden a cosas reales: una parcela, una manzana, una presa — y clasifica los segmentos.

El artículo original usaba una cuadrícula de 100×100 m; el curso evolucionó a la **segmentación de Shepherd** (Shepherd, Bunting & Dymond, 2019). Son **tres subprocesos**, y cada uno arregla lo que el anterior deja pendiente:

1. ① **Siembra (K-means)** — agrupa los píxeles en `numClusters` **familias espectrales** mirando solo su firma, *sin geografía*. Resultado: sal y pimienta (vecinos de la misma manzana caen en familias distintas).
2. ② **Aglomerado (clumping)** — los píxeles *contiguos* de la misma familia se funden en "grupitos". La geografía entra aquí; pero queda confeti de fragmentos diminutos.
3. ③ **Eliminación iterativa** — todo grupo menor a `minSegmentSize` se fusiona, del más chico al más grande, con su vecino *espectralmente más parecido*. Sobreviven solo objetos con tamaño suficiente.

La figura de abajo **no es una caricatura: es este algoritmo corriendo de verdad**, aquí en la página, sobre un paisaje en miniatura como el del taller — cultivos, una **presita de 4 píxeles**, la ciudad y su **parque**. Juega: cambia ① y ③, recorre los pasos y observa los contadores.

Pregunta clave: ¿con qué valor de `minSegmentSize` desaparece la presita?

💡 En el taller usarás `shepherd-wasm` — nuestra implementación validada **bit a bit** contra la referencia — con estos mismos dos parámetros, sobre los 12 millones de píxeles de Aguascalientes.

① numClusters:

3

5

8

③ minSegmentSize:

1 px

4 px

10 px

▶ ver el proceso

la imagen — borde de la ciudad



③ eliminación: 12 objetos finales



ver subproceso:

① familias

② grupitos

③ objetos

5 familias → 22 grupitos → 12 objetos (minSegmentSize=4)

12 objetos finales. ✓ la presita (4 px) sobrevivió como objeto. △ el parque nunca se separó de la ciudad: su firma no consiguió familia propia en el paso ① (sube numClusters). Esa es TU decisión en el taller: ¿qué tamaño tiene el objeto más chico que te importa?

↑ esto no es una caricatura: es el algoritmo de Shepherd corriendo aquí mismo, en tu navegador — igual que lo hará en el taller

5 La verdad-terreno: etiquetas oficiales de INEGI

Para aprender, el modelo necesita **ejemplos confiables**. Usamos el **Marco Geoestadístico del Censo 2020**: los **Polígonos de Localidades Urbanas y Rurales Amanzanas** de todo México.

Detalle fino del artículo: las localidades *rurales amanzanas* no alcanzan la población de "urbanas", pero **su traza de manzanas es idéntica vista desde el satélite** — por eso ambas forman una sola clase **URBANA (1)**; el resto es **NO URBANA (2)**.

Pero ojo — y esta es la idea de la figura: el dato de INEGI y la imagen **hablan idiomas distintos**. El polígono censal es **vectorial**: vértices, coordenadas y una ficha de atributos. La imagen es **ráster**: una malla de píxeles. Para que la verdad-terreno pueda encontrarse con la imagen (y entrar a la tabla de la figura 6), hay que **traducirla**: *rasterizar* el polígono sobre la malla exacta de la imagen. Cada celda hace una sola pregunta — "*¿mi centro cae dentro del polígono?*" — y su respuesta (1 o 2) ya habla idioma píxel. En el taller TÚ harás esta traducción con `rasterize()`, como parte del linaje de tus etiquetas.

6 De la imagen a la tabla

Los algoritmos no comen imágenes: comen **tablas**. Para cada objeto calculamos sus **estadísticas zonales** (media y desviación de cada banda) y su proporción de píxeles urbanos: **una fila por objeto, una columna por característica**. El artículo lo hacía con la *Raster Attribute Table* de RSGISLib; hoy bastan unas líneas de `numpy` que corren hasta en el navegador.

La figura de abajo lo muestra **en vivo, con los objetos de la figura 4**: eliges un objeto, se **intersecta con la imagen**, sus píxeles se **extraen como recorte** (con la forma exacta del objeto), se despliegan sus **6 bandas...** y todo se condensa en **una fila** con números calculados de verdad.

extrae un objeto:

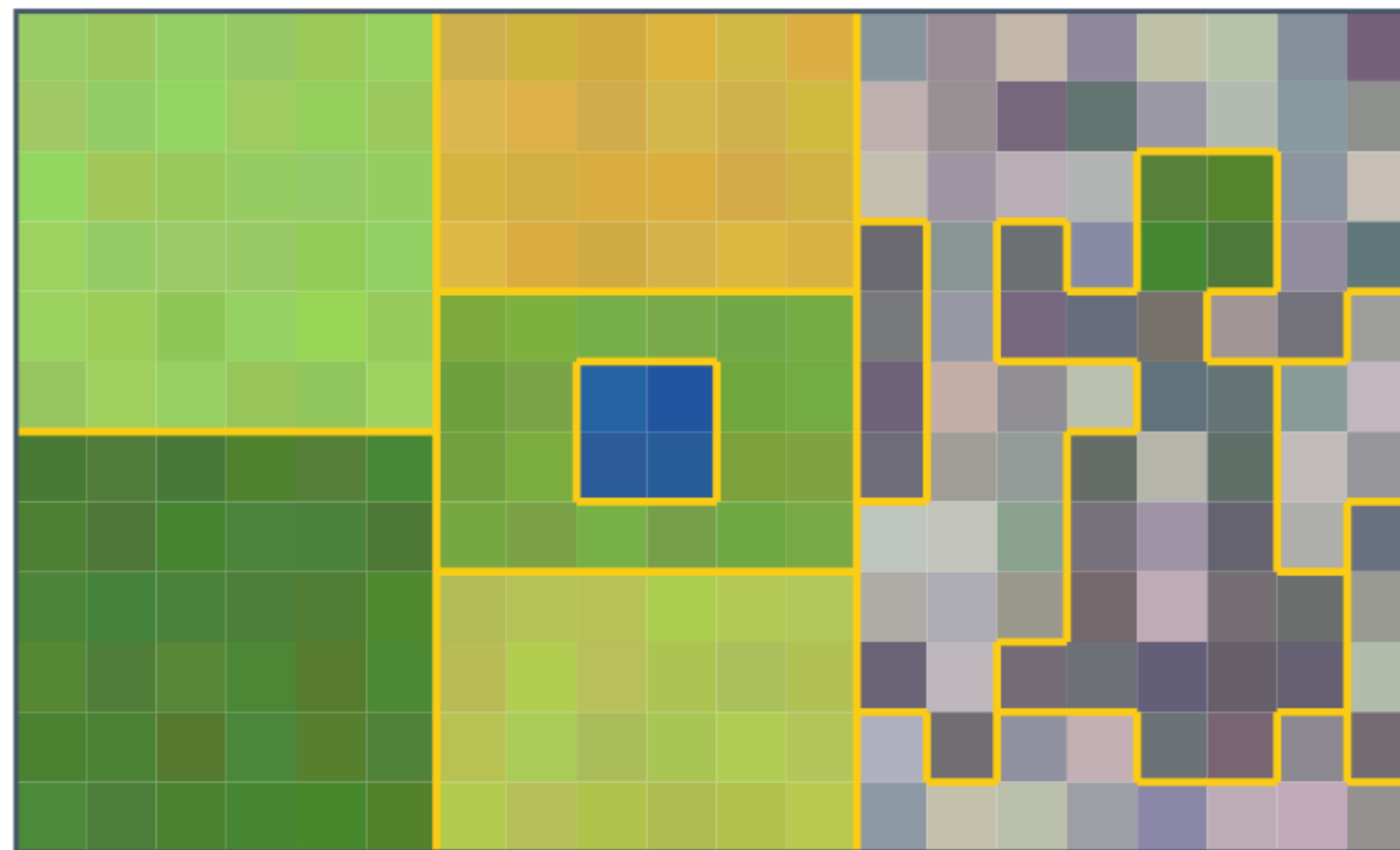
 parcela

 presita

 manzana

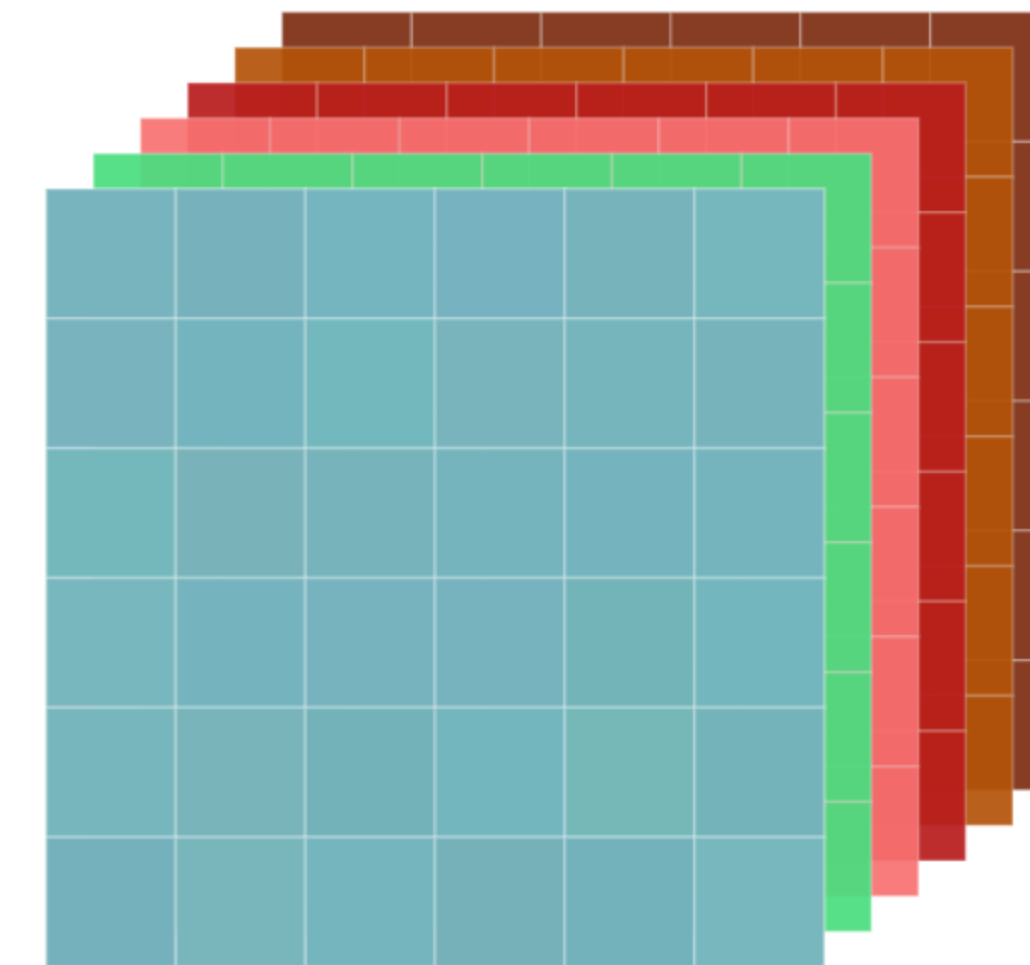
 repetir

los objetos de la figura 4



-  b6 SWIR2
-  b5 SWIR1
-  b4 NIR
-  b3 rojo
-  b2 verde
-  b1 azul

4- la agregación (Σ) resume cada pila en un valor



una banda = una pila de números → la agregación la resume en un valor

banda NIR · 36 píxeles · 0–1

0.88	0.86	0.89	0.86	0.88	0.90
0.87	0.89	0.93	0.89	0.90	0.87
0.93	0.87	0.87	0.88	0.87	0.89
0.92	0.88	0.87	0.87	0.89	0.90
0.91	0.89	0.86	0.90	0.94	0.88
0.85	0.91	0.90	0.86	0.86	0.92

Σ
agregar

Σ suma	31.95
μ media	0.89
σ^2 var	0.001
mín	0.85
máx	0.94

× 6 bandas

$\mu \cdot \sigma^2 \cdot \text{mín} \cdot \text{máx}$

la fila → DataFrame

	n_px	b1 μ	b2 μ	b3 μ	b4 μ	b5 μ	b6 μ
#1	36	0.38	0.80	0.60	0.89	0.71	0.58

b1 μ ...b6 μ = media de cada banda · 1 fila por objeto ✓

Cada banda es una colección de números (intensidad 0–1 por píxel). Una agregación —suma, media, varianza, mínimo o máximo— colapsa esa pila en un solo valor por banda. Así el objeto entero cabe en UNA fila.

↑ la estadística zonal EN VIVO: objeto n imagen → recorte → 6 pilas de números → Σ agregación → una fila (números calculados de verdad)

7 Aprendizaje automático: entrenar y calificar con honestidad

Aprender, para una máquina, es **encontrar patrones en ejemplos etiquetados**. La receta del taller — la misma del artículo:

1. **Depurar:** solo objetos *puros* (≥ 90 % de una clase) — los ambiguos enseñan mal.
2. **Balancear:** mismo número de ejemplos por clase, para no sesgar.
3. **Separar 70/30:** el 30 % queda **bajo llave** — el examen sorpresa que mide la verdad.
4. **Entrenar — el pipeline apilado:** `StandardScaler` pone las 12 características en la misma escala → la red neuronal `MLP` **opina** (sus probabilidades y su voto) → esa opinión **se ANEXA al vector** como características extra (el *StackingEstimator*: de 12 números pasamos a 15) → `ExtraTrees`: **100 árboles votan sobre el vector enriquecido** y la mayoría decide.
5. **Evaluar:** matriz de confusión sobre el 30 % nunca visto. En el artículo: **~94 %**; en tu navegador verás cifras similares.

► recorrer el flujo

De DATOS a CLASIFICACIÓN: el pipeline, paso a paso

La TABLA DE DATOS — el punto de partida (de la figura 6)

Cada fila es un objeto. Para que un algoritmo pueda aprender, la tabla DEBE tener:

- ① **características en NÚMEROS** ($b1\mu$, $b2\mu$, ...) ② **una columna de ETIQUETA** (clase)

$b1\mu$	$b2\mu$	$b3\mu$	...	$b6\mu$	clase
0.38	0.80	0.60	...	0.58	urbano
0.55	0.71	0.49	...	0.62	no urb
0.44	0.46	0.44	...	0.39	urbano

↓ esta tabla alimenta el algoritmo

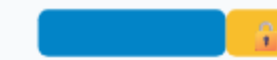
- 1 Depurar: quedarse solo con objetos PUROS ($\geq 90\%$ de una clase)**
los objetos ambiguos enseñan mal — se descartan para tener ejemplos confiables



- 2 Balancear: mismo número de ejemplos de cada clase**
así el modelo no se inclina hacia la clase más abundante



- 3 Separar 70 / 30: enseñar vs. examen**
70 % para que el modelo aprenda · 30 % guardado: NUNCA lo ve al entrenar



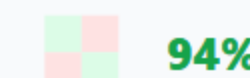
- 4 Escalar: poner las características en la misma escala (StandardScaler)**
ninguna banda domina solo por tener números más grandes



- 5 Entrenar: el modelo aprende los patrones**
la red neuronal (MLP) opina → su opinión se ANEXA (stacking) → 100 árboles (ExtraTrees) votan la clase de cada objeto



- 6 Evaluar: calificar con el 30 % guardado**
matriz de confusión + exactitud (~94 %): qué tan bien generaliza



- 7 Clasificar TODO: aplicar el modelo a cada objeto → el MAPA**
de una tabla de datos a un mapa de decisiones: urbano / no urbano



datos → clasificación ✓

8 ¿Qué es un cuaderno Jupyter?

Un **cuaderno Jupyter** es la libreta de laboratorio de la ciencia de datos: una página viva hecha de **celdas**. Unas celdas son texto (como esta página); otras son **código Python que se ejecuta** con Shift+Enter, mostrando su resultado justo debajo — un número, una tabla, un mapa.

Eso lo vuelve perfecto para aprender: lees la explicación, ejecutas la celda, **ves el efecto al instante**, cambias un parámetro y vuelves a ejecutar. El taller entero es un cuaderno; esta página imita su forma para que llegues entrenado.

▶ Ejecutar la celda actual

⏏ Ejecutar todas

🔄 Reiniciar kernel

Taller_ML.ipynb

      Código ▾ ● Idle

Paso 1: leer la imagen Markdown

```
img = rasterio.open("ags.tif").read()

seg = shepherd(img); plt.imshow(seg)
```

Código · seleccionada

Kernel: Python 3 (Pyodide)

Guía rápida de Jupyter

▶ / Shift + Enter

ejecuta la celda actual y avanza

Ctrl + Enter

ejecuta y se queda en la celda

[] → [*] → [1]

vacía · corriendo · orden de ejecución

⏏ Ejecutar todas

corre el cuaderno de arriba abajo

Markdown / Código

celdas de texto y de Python

Kernel = el motor

🔄 Reiniciar borra todo y empieza de cero

↑ pruébalo: ejecuta la celda actual, ejecuta todas o reinicia el kernel — y observa el contador []→[*]→[N] y el estado del kernel

9 ¿Y dónde corre Python? Opción A: tu navegador (WebAssembly)

En 2023 este ejercicio exigía instalar Conda, GDAL y RSGISLib — horas de configuración y permisos. Hoy existe **WebAssembly (WASM)**: un formato que permite ejecutar programas completos **dentro del navegador**, en una "caja de arena" segura. El proyecto **Pyodide** compiló Python científico entero a WASM: numpy, rasterio, scikit-learn... corriendo en la pestaña.

¿Es una "caja de arena" **de verdad**? Sí — y vale la pena entender la anatomía, porque responde todas las dudas prácticas del taller: **el servidor solo entrega copias y es de solo lectura** (nadie puede escribirle); el Python del taller corre **encerrado en la pestaña** (no puede ver tu disco C:\ ni tus otras pestañas); tus archivos viven en **dos casas distintas** — la *memoria del kernel* (/tmp, que se esfuma al cerrar) y el *almacén del navegador* (IndexedDB, donde sobrevive TU copia del cuaderno); y entre la pestaña y tu disco hay un muro que **solo cruzan los archivos que TÚ subes o descargas con un clic**.

Y la pregunta que todos se hacen tarde o temprano: **¿qué pasa si borro TODOS los archivos del explorador del Lab?** Pulsa los escenarios de la figura y descúbrelo — la respuesta es liberadora.

escenarios:

▶ abres el taller

✎ guardas (Ctrl+S)

✖ cierras la pestaña

🗑 ¡borras TODO!

🔒 ¿y mi disco C:\?

el servidor

GitHub Pages / kit

📄 Taller.ipynb

📁 animaciones

🌐 imagen estatal



**SOLO LECTURA: nadie
puede escribirle ni dañarlo**

TU navegador

la pestaña · sandbox WASM

🐍 **Python (wasm32)**

numpy · rasterio · sklearn · shepherd

💾 **/tmp · memoria del kernel**

la imagen (176 MB) vive aquí

almacén del navegador

(IndexedDB · sobrevive a recargas)

📄 TU copia del cuaderno

solo TU navegador, TU perfil:
nadie más lo ve

↑↓
TU clic

tu disco (C:\) — el sandbox NO puede verlo ni tocarlo

El servidor entrega UNA copia de los originales y ahí termina su papel. Desde este momento trabajas sobre TU copia, dentro de tu navegador. El servidor es de solo lectura: jamás se entera de lo que haces.

↑ pulsa los 5 escenarios — el último responde la gran pregunta: borrar TODO no rompe nada

10 Opción B: el laboratorio portable (cuando los datos crecen)

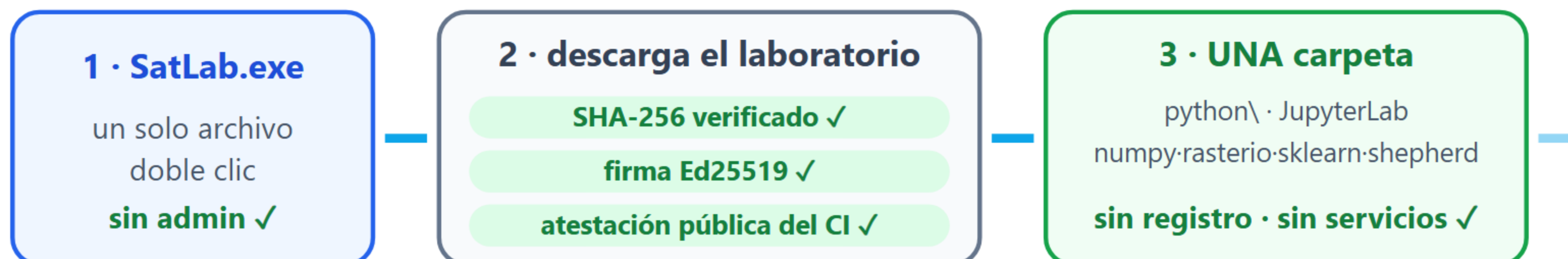
El navegador es imbatible para aprender — y con la imagen calidad Landsat procesa **el estado completo**. Pero el siguiente nivel de detalle, la geomediana Sentinel-2 de 10 m y 12 bandas, pesa **2.66 GB**: más que el techo de memoria de una pestaña. Para esas sesiones entregamos **SatLab**: un solo ejecutable que instala en una carpeta un JupyterLab idéntico, con todo el stack verificado criptográficamente — y desde el cual puedes además **descargar imágenes frescas de tu zona con Google Earth Engine**. **El mismo cuaderno corre en ambos** — solo cambia el motor debajo.

	 Navegador (WASM)	 Portable (SatLab)
Instalación	ninguna — abrir una URL	descargar 1 ejecutable + 1 clic
Permisos de administrador	no	no
Tamaño de datos	el estado a 30 m (calidad Landsat)	Sentinel-2 completa a 10 m (GB)
Velocidad	buena (≈ nativa en tiles)	máxima + todos tus núcleos
Sin internet (después de instalar)	requiere cargar la página	funciona 100% sin conexión
Acceso a tu disco	no (sandbox sellado)	sí — su gran poder, con responsabilidad
Autorización de TI	no se requiere (es una página web)	sí: consúltala antes de instalar
Huella al desinstalar	cerrar la pestaña	borrar la carpeta

Transparencia primero. A diferencia del navegador, el portable instala un **Python real** en tu máquina — software nativo, de la misma familia que QGIS u Office. Eso es exactamente lo que le da su poder (gigabytes de datos, todos tus núcleos, acceso a tus carpetas)... y también lo que implica una **exposición potencial** que el sandbox no tiene: un Python sin control puede representar riesgos de seguridad. Por eso la regla del curso es clara: **en equipo institucional, solicita la autorización de tu área informática antes de instalarlo** y revisa la estrategia interna de tu institución. Para facilitar ese diálogo, el proyecto publica su **cadena de verificación completa** — hashes SHA-256, firma Ed25519 y atestación pública del proceso de construcción — con una [guía dirigida a las áreas de TI](#).

✦ La regla del curso: **aprende en el navegador, produce en el portable** — con la bendición de tu área de TI. Mismo Python, mismos cuadernos, misma ciencia.

la arquitectura del portable, paso a paso



el contraste honesto: mismo cuaderno, distinto alcance

 **NAVEGADOR · caja sellada**

no puede ver tu disco — sandbox de verdad
aprende y procesa el estado a 30 m

 **exposición: prácticamente nula**

 **PORTABLE · Python real**

Sí ve tu disco: GB de datos, todos tus núcleos,
Google Earth Engine — ese es su PODER

 **y también una exposición real: es software nativo**

En equipo institucional: pide la autorización de tu área de TI

Un Python instalado sin control puede representar riesgos de seguridad — revisa la estrategia interna de tu institución.

Para facilitar el diálogo, llévale a TI la cadena de verificación del proyecto (hashes, firma y atestación del CI):

github.com/abxda/portable-satelital → VERIFICACION.md (guía para áreas de TI)

El portable no se esconde: poder real, exposición declarada y una cadena de verificación lista para auditarse. La decisión es de tu institución.

11 El mapa completo del taller

Paso	Concepto	Herramienta
1	Imagen limpia anual	Geomediana Sentinel-2 (Zenodo)
2	Del píxel al objeto	shepherd-wasm
3	Verdad-terreno	Marco Geoestadístico INEGI 2020
4	Imagen → tabla	numpy
5	Entrenar y evaluar (el corazón)	scikit-learn (MLP + ExtraTrees, 70/30)
6	Mapa clasificado + descargas	geopandas + matplotlib

 **Abrir el taller de Machine Learning**

← **Portada**

el viaje completo: de la luz del Sol al mapa de decisión



en el navegador hoy · en el portable cuando tu institución lo autorice · con TUS datos cuando quieras

El método completo de un vistazo: síguelo aquí, ejecútalo en el taller, repítelo con tu territorio.